

GoalForge — Current State Review Report

Date: 2026-03-14

Reviewer roles: Lead Product Manager + Senior Software Engineer

Prior review date: 2026-03-13 (PR #24)

Scope Reviewed

- Backend: apps/api (FastAPI, auth, AI orchestration, data model, tests)
- Frontend: apps/web (React/Vite UX, state management, API integration)
- CI and project setup: .github/workflows/ci.yml, root and app READMEs

Changes Since Previous Review (2026-03-13)

The following areas have changed or been updated since the prior agent session:

- Auth exception handling partially hardened — `_decode_token` in `auth.py` now catches `jwt.ExpiredSignatureError` and `jwt.InvalidTokenError` specifically, in addition to the existing broad fallback.
- Dependency updates — Frontend dependencies updated: React 19, Vite 7, TypeScript 5.9, Tailwind CSS v4, axios 1.13.6.
- No new frontend tests added — The testing gap identified in the prior review remains.
- `Dashboard.tsx` remains monolithic — 951 lines with no decomposition.
- No pagination added — Goals listing API still returns all records without limits.

New findings discovered in this review are identified with [NEW] below.

1) Lead Product Manager Review

Product Strengths

- Clear core value proposition — GoalForge has a differentiated loop: AI-generated SMART goals + sprint tasks + gamification.
- Coherent motivational system — Star points, streaks, evolution stages, and 'Hall of Fame' are implemented end-to-end.
- Foundational user journey exists — Sign-in/sign-up, dashboard, analytics, and protected routes are in place.
- [NEW] Active dependency maintenance — Frontend dependencies are kept current (React 19, Vite 7, TypeScript 5.9).

Product Risks / Gaps

- High feature concentration in one UI file slows iteration velocity — `Dashboard.tsx` is 951 lines, increasing regression risk.
- No frontend automated tests — There are backend tests but no frontend tests in `apps/web/src`.
- Scalability gap in goal listing API — Goals API returns all goals without pagination.
- Operational reliability mismatch between local and CI defaults — Local tests need `RATE_LIMIT_ENABLED=false`.

⚠ [NEW] P0 BUG — Silent sprint generation failure on milestone advance

In `complete_milestone` (`apps/api/main.py`, line 657), synchronous sprint task generation catches `ValueError` but

`generate_sprint_tasks` raises `AIGenerationError`. This mismatch means AI generation errors bubble up as unhandled 500 responses rather than actionable 502 error messages.

PM Priority Recommendations

P0 (next sprint)

- Fix the wrong exception class in `complete_milestone` (`ValueError` -> `AIGenerationError`).
- Add frontend test baseline for critical flows (goal creation, task completion, status update).
- Split Dashboard into composable feature components to reduce delivery risk.

P1

- Add paginated goals listing and front-end support.
- Add explicit failure UX for profile/AI-related fetch degradation.
- Migrate FastAPI startup handler from deprecated `on_event` to lifespan pattern.

P2

- Define product analytics events (activation, weekly retention, milestone completion rate).

2) Senior Software Engineer Review

Engineering Strengths

- Clean backend layering and typed contracts — Good separation of concerns across `main.py`, `auth.py`, `ai_utils.py`, `schemas.py`, and `models.py`.
- [NEW] Improved auth exception specificity — `_decode_token` now explicitly catches `jwt.ExpiredSignatureError` and `jwt.InvalidTokenError`.
- Reasonable security foundations — JWT verification with JWKS fetch/caching; auth checks enforced per route; CORS configured from env.
- AI output schema constraints are present — Gemini responses are validated through Pydantic schemas.
- Backend test suite is healthy — 23 tests, all passing.
- [NEW] CI pipeline is comprehensive — Backend tests plus frontend lint, TypeScript type check, and build on every push and PR.

Engineering Issues and Improvement Opportunities

⚠ [NEW] BUG: Wrong exception caught in `complete_milestone`

At `apps/api/main.py` line 657, `'except ValueError'` should be `'except AIGenerationError'`. `generate_sprint_tasks` raises `AIGenerationError` (`exceptions.py`), never `ValueError`. Sync generation failures propagate as unhandled 500s.

```
# Current (broken):
except ValueError as exc:
    raise HTTPException(status_code=502, detail=str(exc))
```

```
# Should be:
except AIGenerationError as exc:
    raise HTTPException(status_code=502, detail=str(exc))
```

- `auth.py` retains broad fallback exception in token decode path — A final broad `'except Exception'` fallback remains in `_decode_token` (lines 105-109).

⚠ [NEW] Deprecated FastAPI startup event handler

`@app.on_event('startup')` in `main.py` (line 94) is deprecated in FastAPI and emits `DeprecationWarning` in test output. Replace with lifespan context manager pattern.

- Rate-limit keying strategy can cause noisy-neighbor effects.
- Frontend reliability and maintainability pressure — Monolithic `Dashboard.tsx` (951 lines) with mixed concerns.
- API scalability and resilience — No pagination in goals listing; no durable retry queue for background tasks.
- Test and quality asymmetry — Backend has tests; frontend has none.

Recommended Technical Roadmap

P0 hardening (immediate)

- Fix `complete_milestone`: change `'except ValueError'` to `'except AIGenerationError'`.
- Remove or scope the remaining broad `'except Exception'` fallback in `_decode_token`; add structured logging.
- Add frontend tests for highest-risk interactions.

P1 architecture

- Migrate `@app.on_event('startup')` to FastAPI lifespan pattern.
- Refactor `Dashboard.tsx` into domain-focused components/hooks.
- Introduce shared data-fetch hooks to remove duplication between Dashboard and Analytics.

P2 performance/scale

- Add cursor/offset pagination for `/users/{user_id}/goals`.
- Add observability metrics for AI generation latency/failure rate.

3) Current Validation Snapshot

Commands executed in this review (2026-03-14):

Frontend baseline

- `npm ci`
- `npm run lint` — PASS (0 warnings)
- `npx tsc --noEmit` — PASS
- `npm run build` — PASS (323 KB JS bundle, gzip 97 KB)

Backend baseline

- `python -m pytest tests/ -v` (with `RATE_LIMIT_ENABLED=false` `GEMINI_API_KEY=dummy`)

Observed status

- Frontend lint/type-check/build: PASS
- Backend tests: 23/23 passed (2 deprecation warnings — `on_event` startup handler)

4) Executive Summary

GoalForge is in a strong MVP-to-beta shape: coherent product loop, good backend structure, and a comprehensive CI pipeline.

Since the previous review (2026-03-13), the most notable improvement is partial auth exception specificity in `_decode_token`. However, one new P0 bug was discovered: `complete_milestone` catches the wrong exception

class (ValueError instead of AIGenerationError), meaning synchronous sprint generation failures during milestone advancement surface as unhandled 500 errors rather than the intended actionable 502 responses.

The highest-leverage next work remains:

- P0 bug fix — correct exception class in complete_milestone
- Frontend quality uplift (tests + dashboard decomposition)
- Security/observability hardening (remove remaining broad auth fallback, migrate startup handler)
- Scalability guardrails (pagination + improved async task reliability)

All improvements can be delivered incrementally without removing existing shipped behavior.